

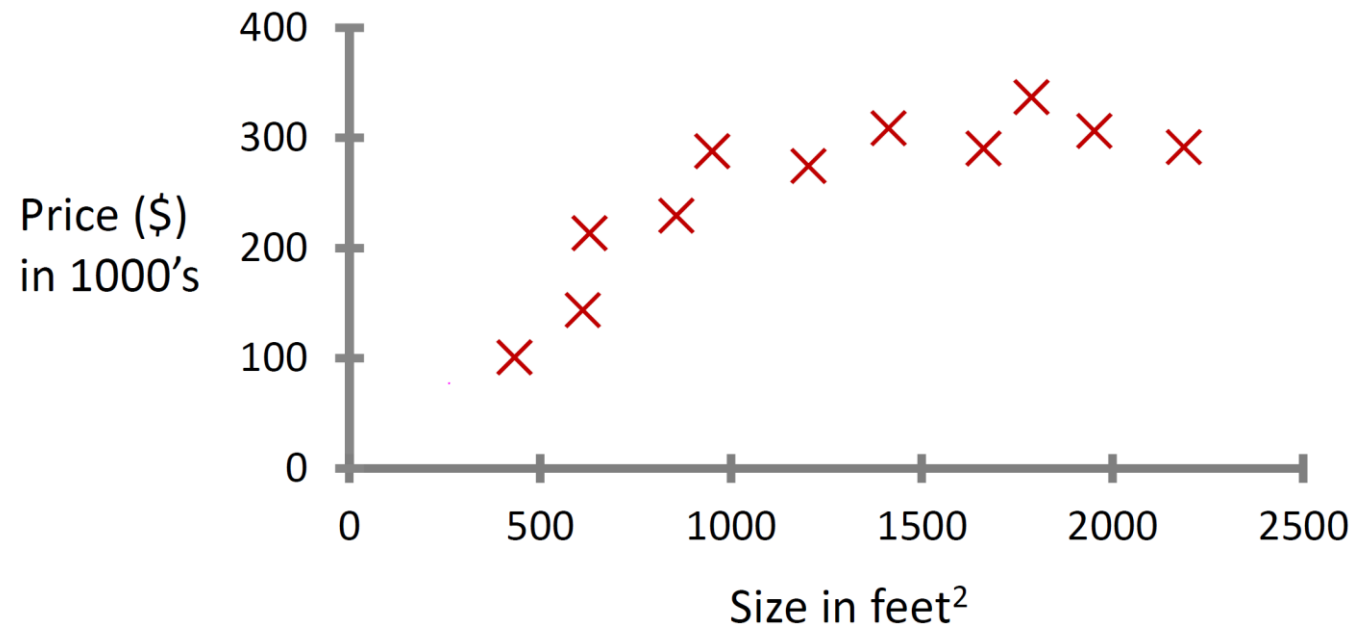
Regression

Prof. Giuseppe Serra

Università degli Studi di Udine

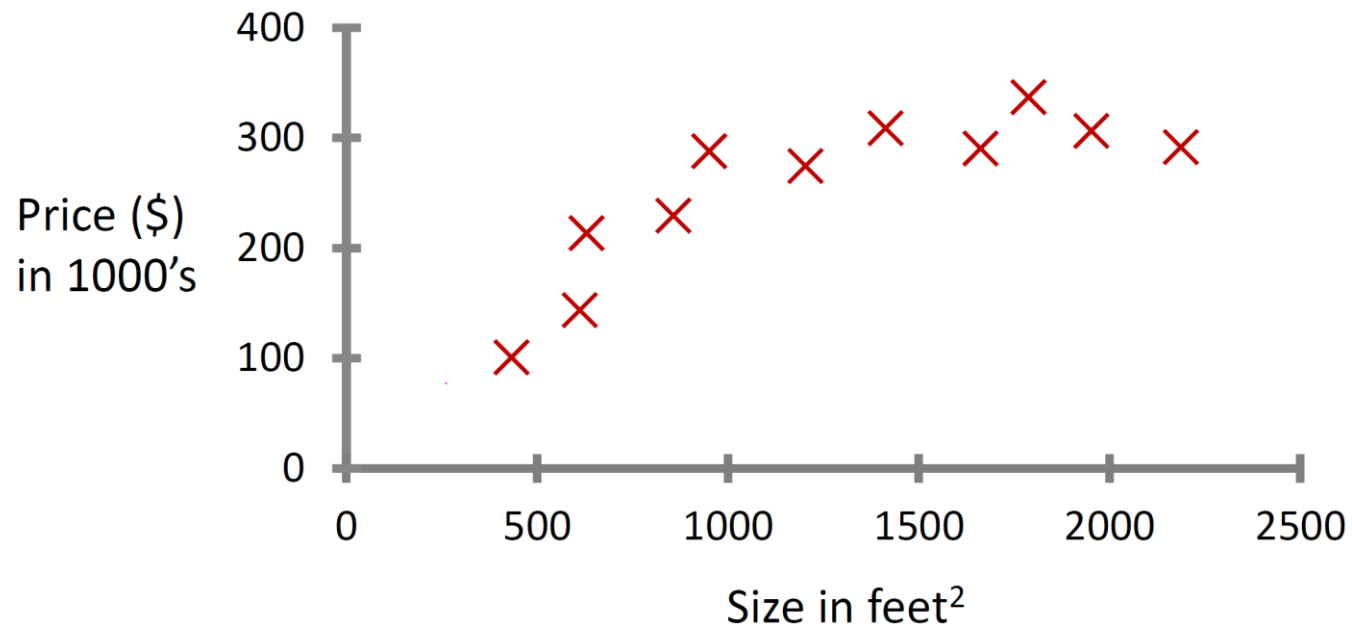
Housing price prediction

Suppose you have a dataset:



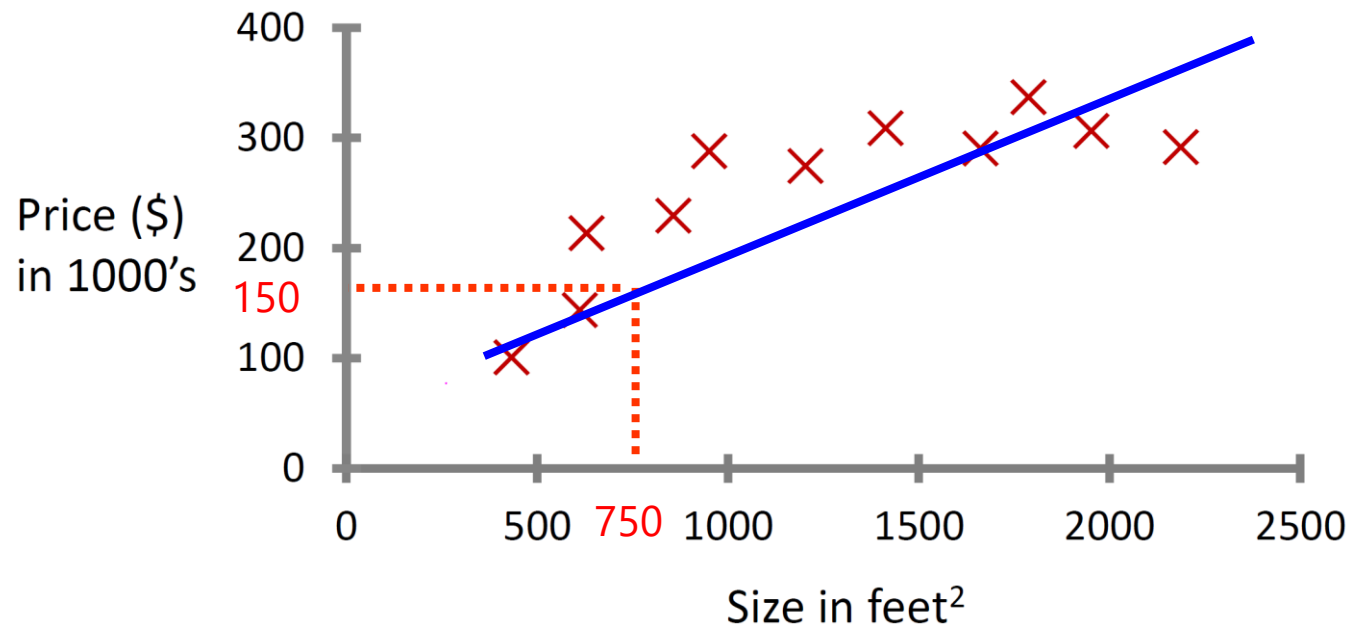
Housing price prediction

Suppose you have a friend with a house of 750 feet². How can a learning algorithm help you?



Housing price prediction

Suppose you have a friend with a house of 750 feet². How can a learning algorithm help you?



A learning algorithm can fit a straight line to the data. Based on that, it looks like maybe the house can be sold for about \$ 150,000.

Supervised Learning

The supervised learning refers to the fact that we use a dataset in which “the right answer” are given.

Dataset

Size in feet ² (x)	Price (\$) in 1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

This algorithm is called regression algorithm, which given an input value (size in feet) it predicts a continuous value as output (price)

Supervised Learning

The supervised learning refers to the fact that we use a dataset in which "the right answer" are given.

Dataset

Size in feet ² (x)	Price (\$) in 1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

m = Number of training examples

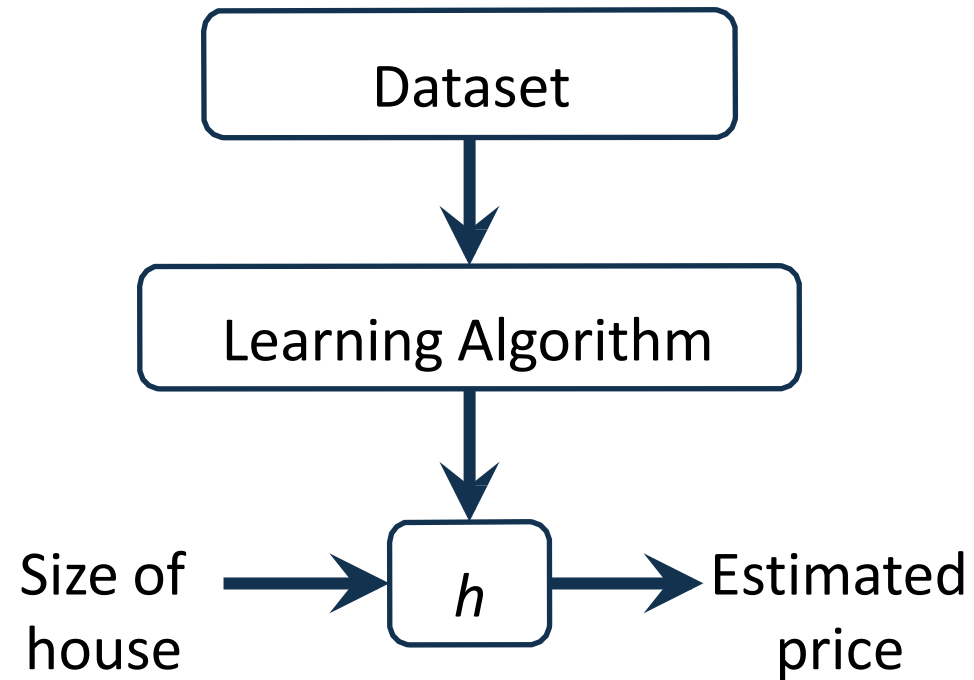
x 's = input variable / features

y 's = «output» variable / «target» variable

(x, y) = one training example

$(x^{(i)}, y^{(i)})$ = i-th training example ...($x^{(1)} = 2104$; $x^{(2)} = 1416$...)

Supervised Learning



h = hypothesis

Linear Regression

How do we represent h?

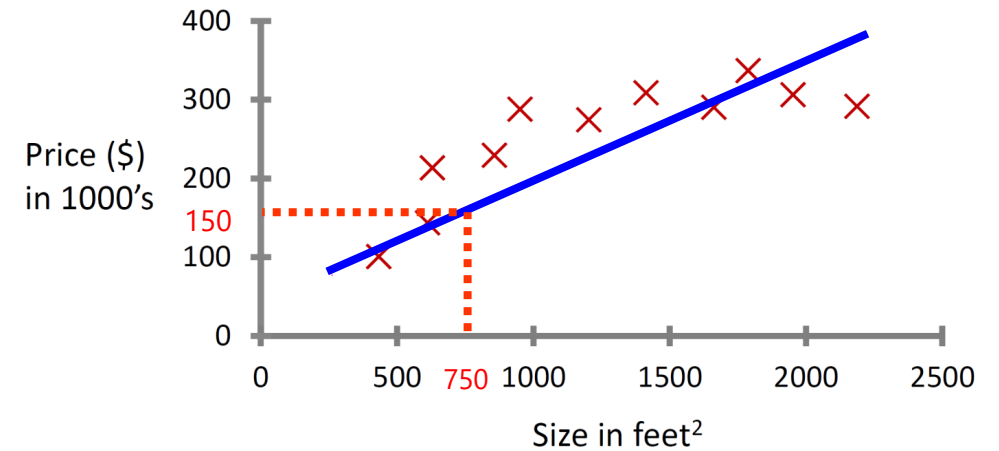
Let's start with a **Linear Regression** with one variable

$$h_{\theta}(x_1) = \theta_0 + \theta_1 x_1$$

For convenience of notation $x_0 = 1$

$$\text{Let's } \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} \quad x = \begin{bmatrix} 1 \\ x_1 \end{bmatrix}$$

The hypothesis can be written in vectorization form: $h_{\theta}(x) = \theta^T x$



Linear Regression

Size in feet2 (x)	Price (\$) in 1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

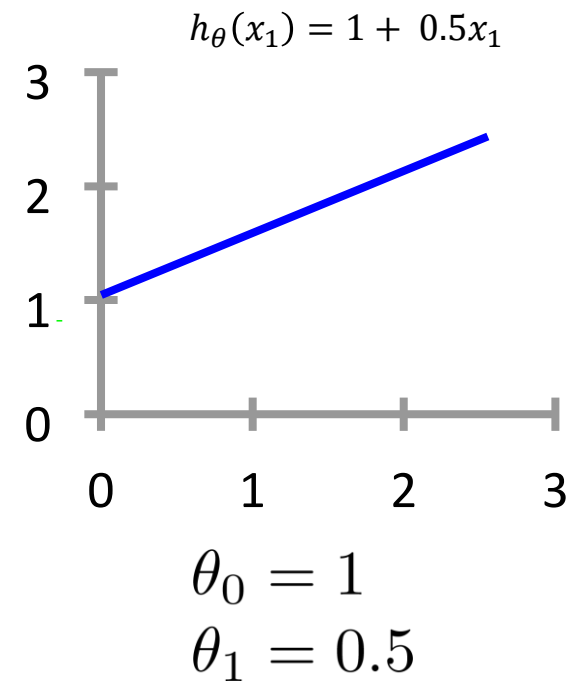
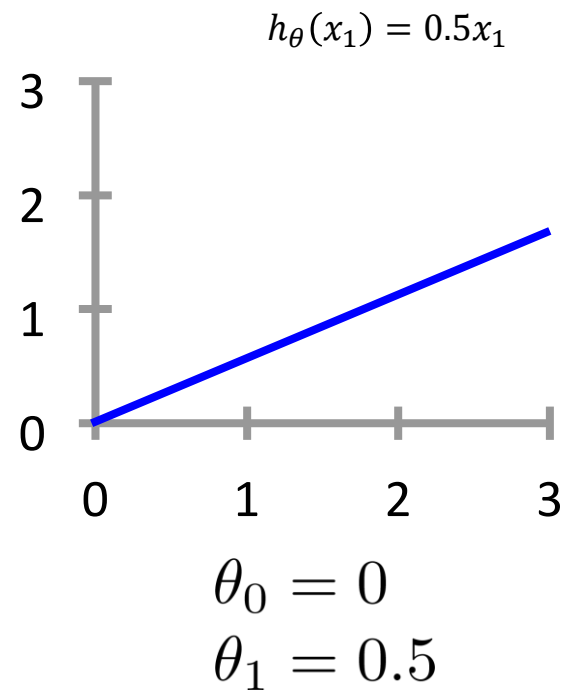
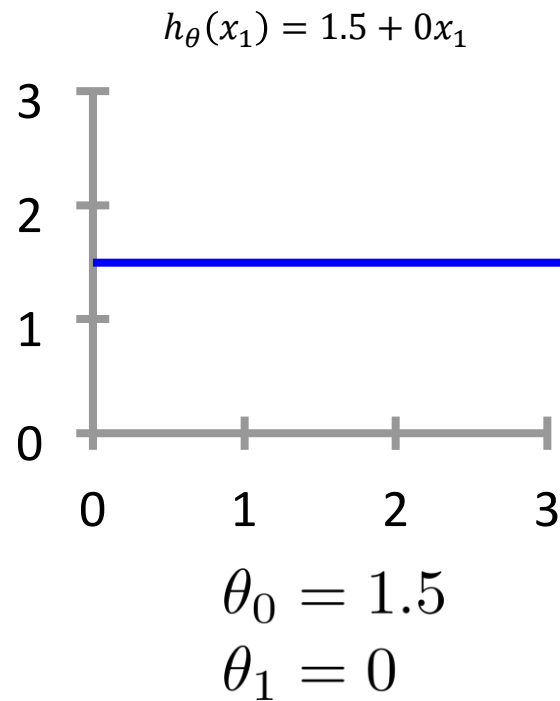
Hypothesis: $h_{\theta}(x_1) = \theta_0 + \theta_1 x_1$

θ_i : Parameters

How to choose θ_i ?

Linear Regression

$$h_{\theta}(x_1) = \theta_0 + \theta_1 x_1$$



Linear Regression

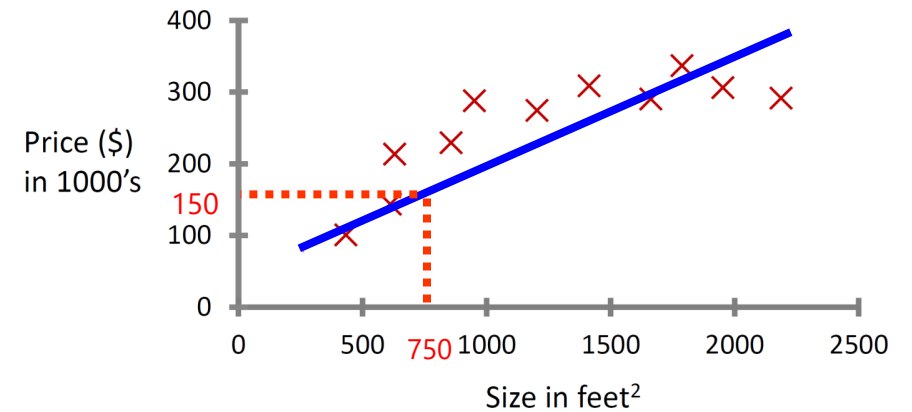
Idea: Choose θ_0, θ_1 so that $h_\theta(x)$ is close to y for our training examples (x, y)

Minimization problem:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)})^2$$

$$\min_{(\theta_0, \theta_1)} J(\theta_0, \theta_1)$$

$J(\theta_0, \theta_1)$ is called **Cost function** (in this example it is also called "squared error cost function")



Cost Function: Intuition I

Hypothesis:

$$h_{\theta}(x_1) = \theta_0 + \theta_1 x_1$$

Parameters:

$$\theta_0, \theta_1$$

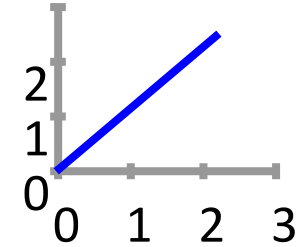
Cost Function

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Goal:

$$\min_{(\theta_0, \theta_1)} J(\theta_0, \theta_1)$$

Simplified



Hypothesis:

$$h_{\theta}(x_1) = \theta_1 x_1$$

Parameters:

$$\theta_1$$

Cost Function

$$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

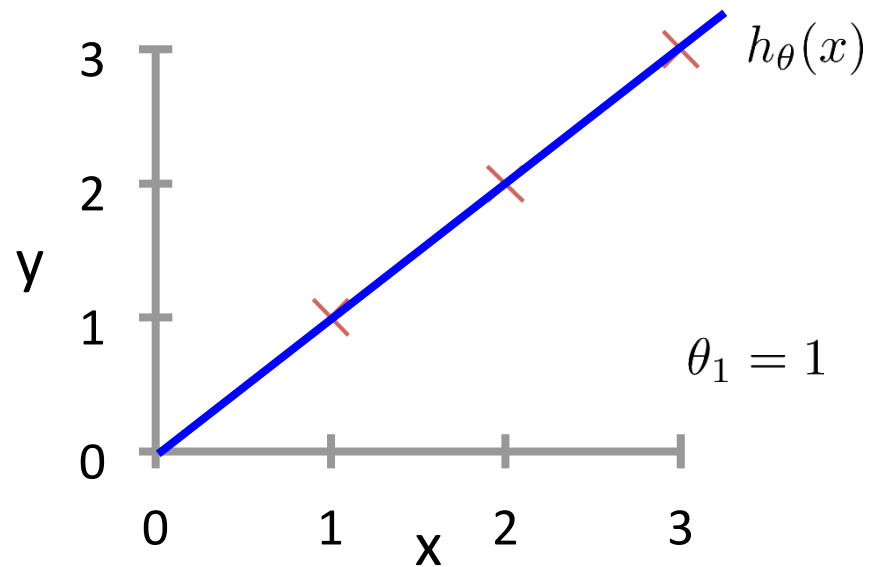
Goal:

$$\min_{\theta_1} J(\theta_1)$$

Cost Function: Intuition I

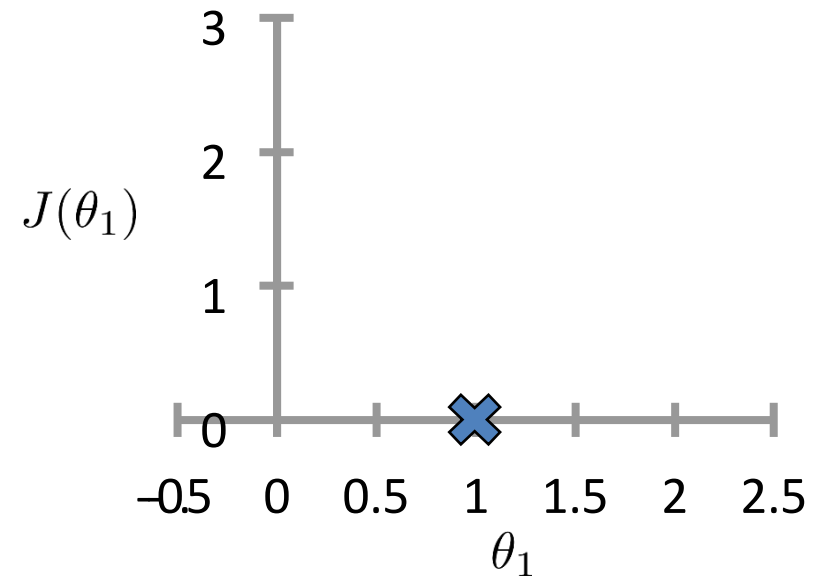
$$h_{\theta}(x)$$

(for fixed θ_1 , this is a function of x)



$$J(\theta_1)$$

(function of the parameter θ_1)

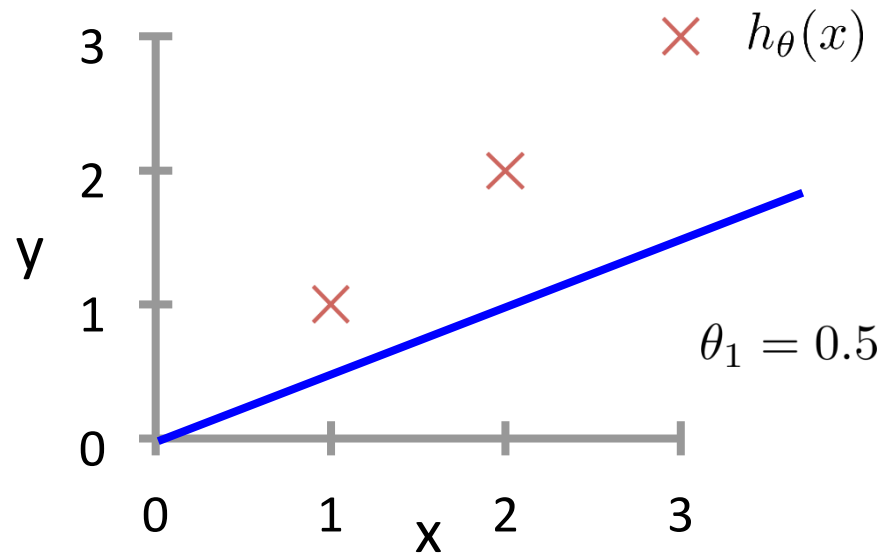


$$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 = \frac{1}{2m} \sum_{i=1}^m (\theta_1 x^{(i)} - y^{(i)})^2 = \frac{1}{2m} (0^2 + 0^2 + 0^2) = 0$$

Cost Function: Intuition I

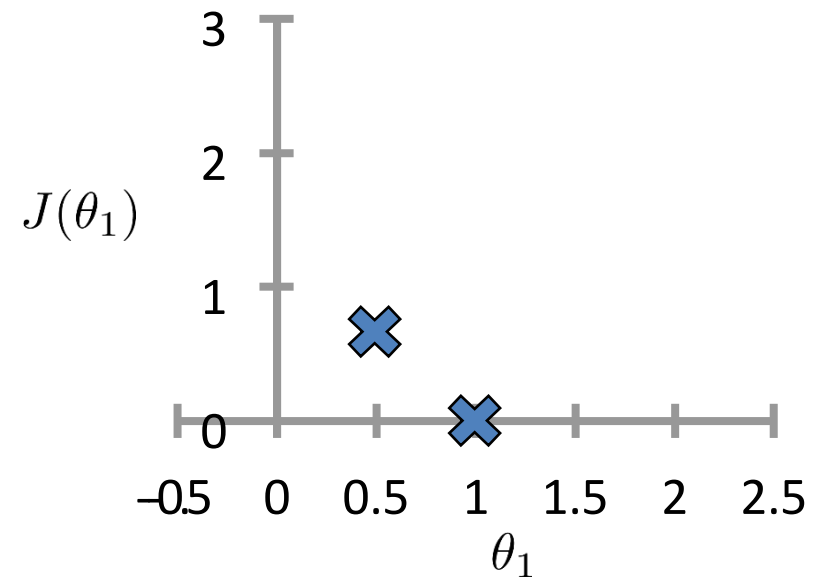
$$h_{\theta}(x)$$

(for fixed θ_1 , this is a function of x)



$$J(\theta_1)$$

(function of the parameter θ_1)

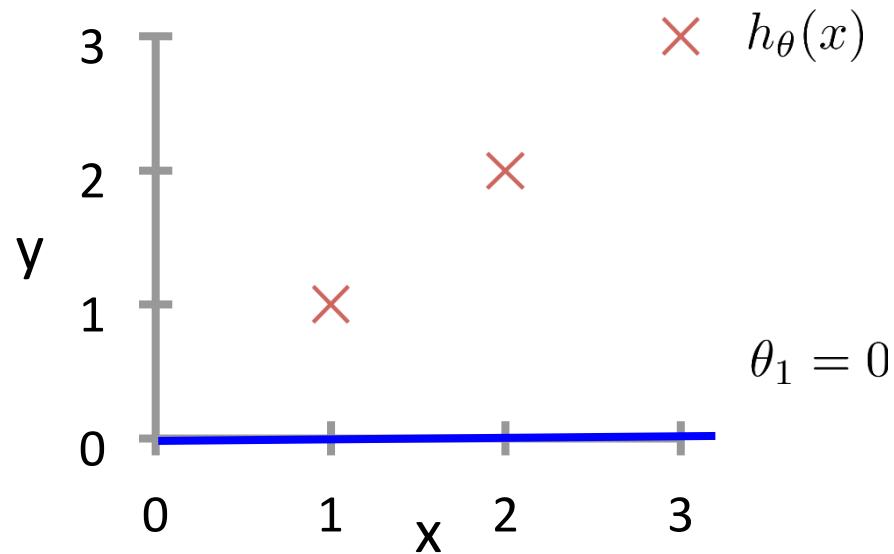


$$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 = \frac{1}{2m} [(0.5 - 1)^2 + (1 - 2)^2 + (1.5 - 3)^2] = 0.58$$

Cost Function: Intuition I

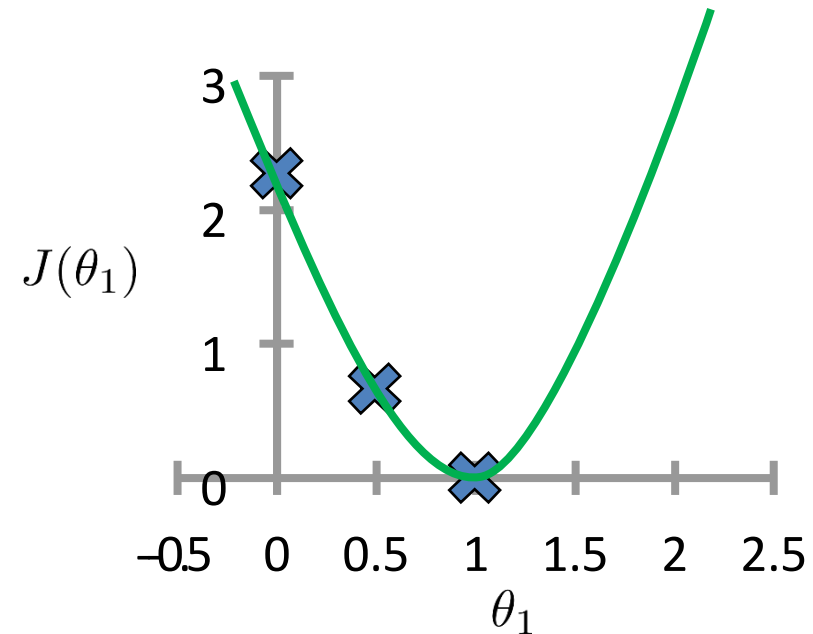
$$h_{\theta}(x)$$

(for fixed θ_1 , this is a function of x)



$$J(\theta_1)$$

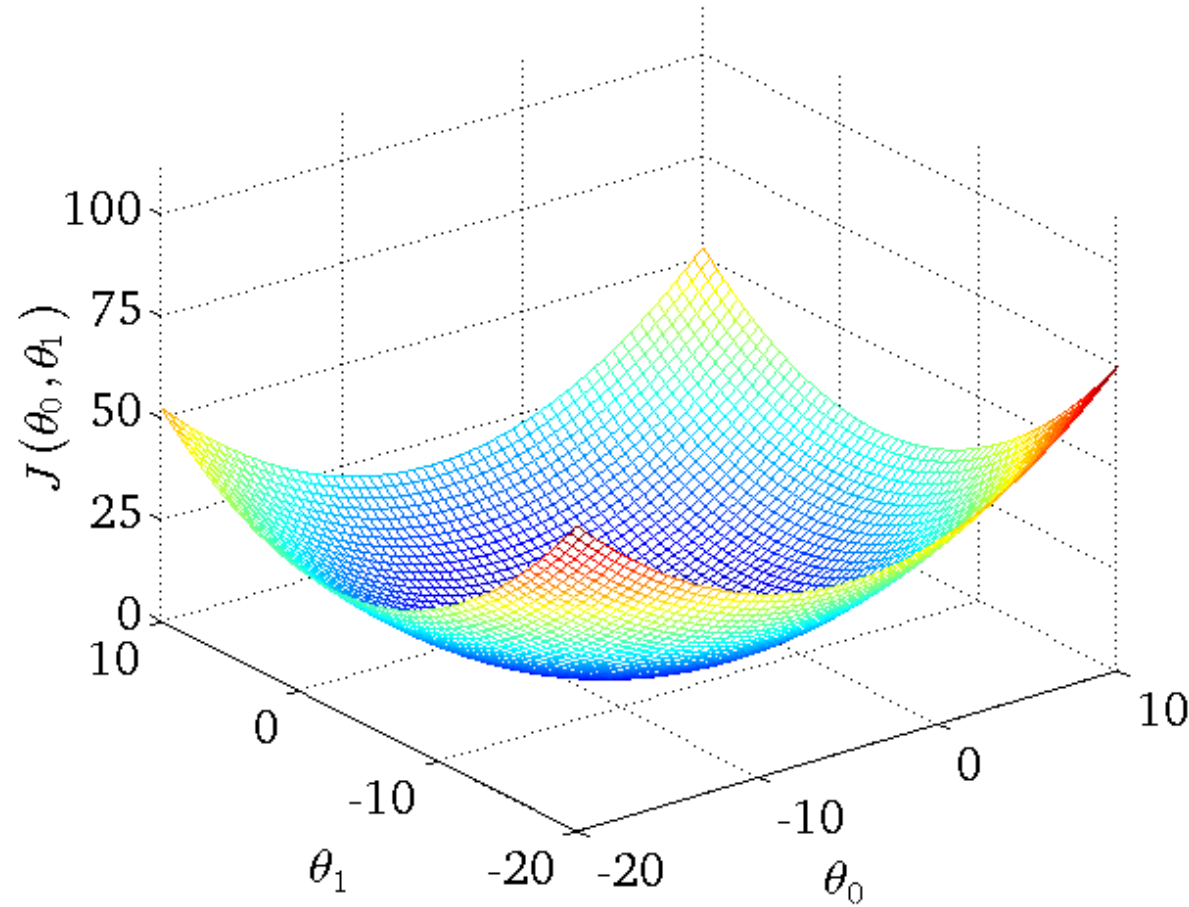
(function of the parameter θ_1)



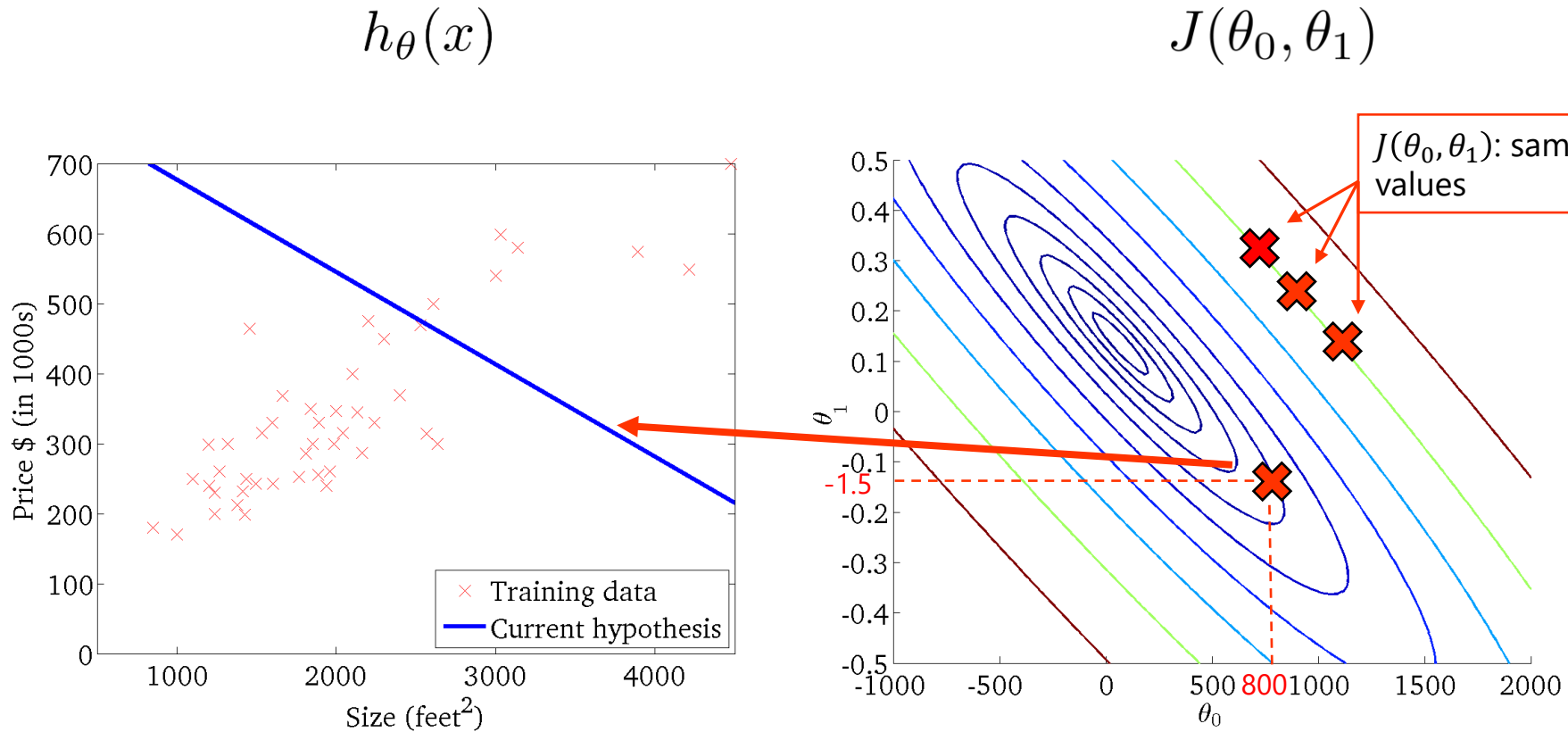
$$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 = \frac{1}{2m} [1^2 + 2^2 + 3^2] = 2.3$$

Cost Function

$$h_{\theta}(x_1) = \theta_0 + \theta_1 x_1$$

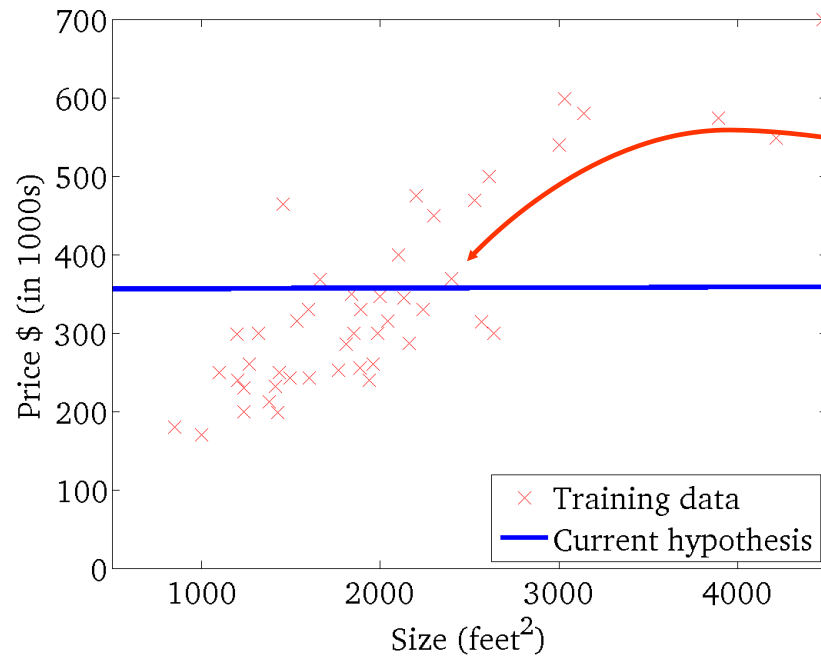


Contour Plots o Linee di livello

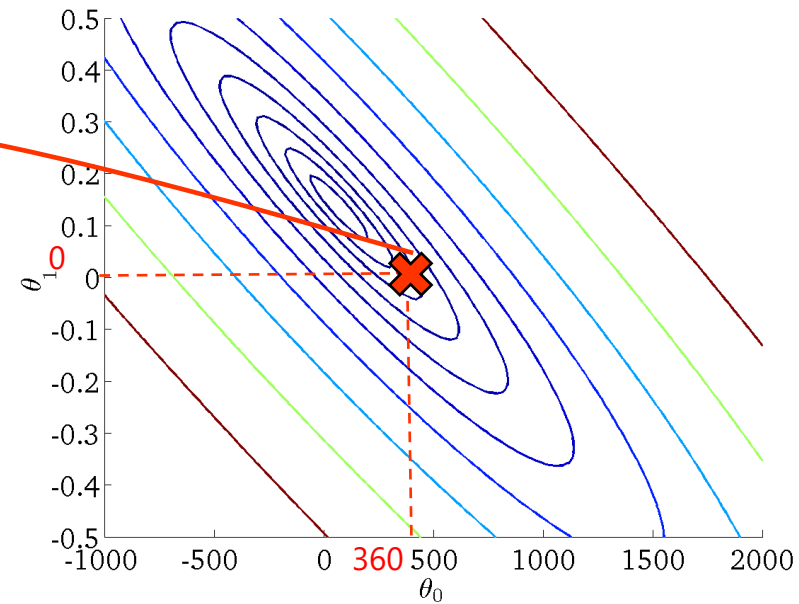


Contour Plots o Linee di livello

$$h_{\theta}(x)$$

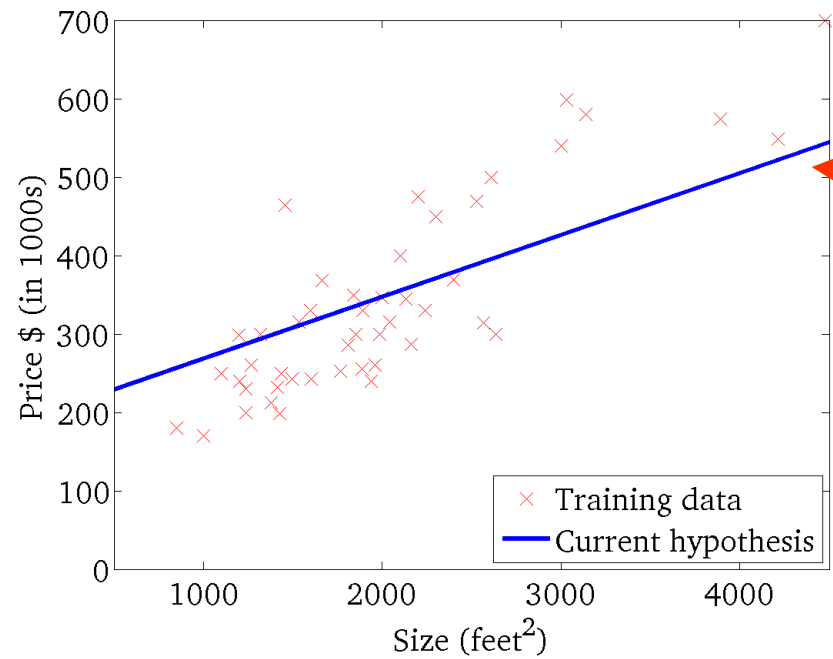


$$J(\theta_0, \theta_1)$$

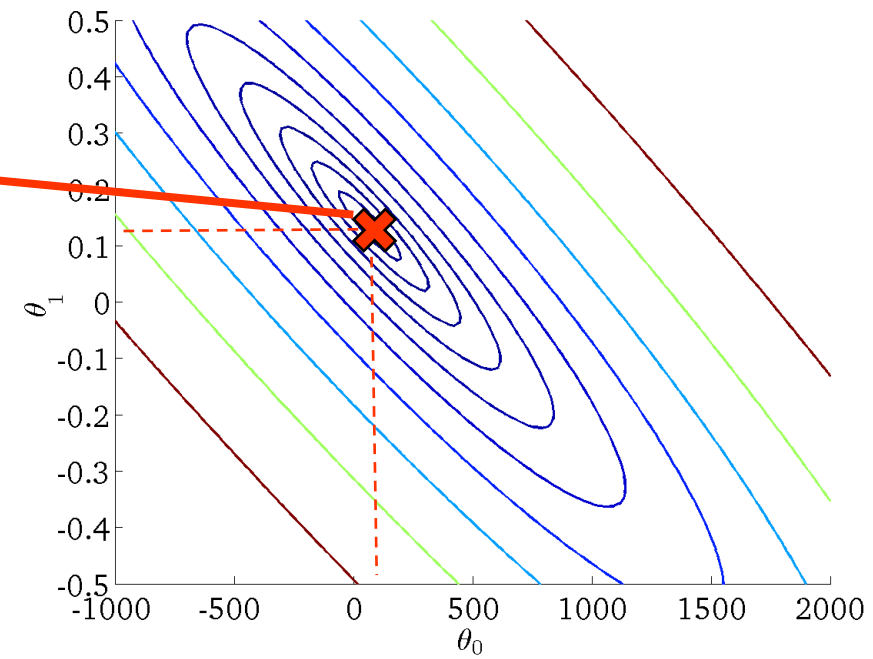


Contour Plots o Linee di livello

$$h_{\theta}(x)$$



$$J(\theta_0, \theta_1)$$



Gradient Descent Algorithm

Gradient Descent Algorithm

Gradient Descent algorithm

GD is a general algorithm to minimize function. Here we are using to linear regression.

Have some function $J(\theta_0, \theta_1)$

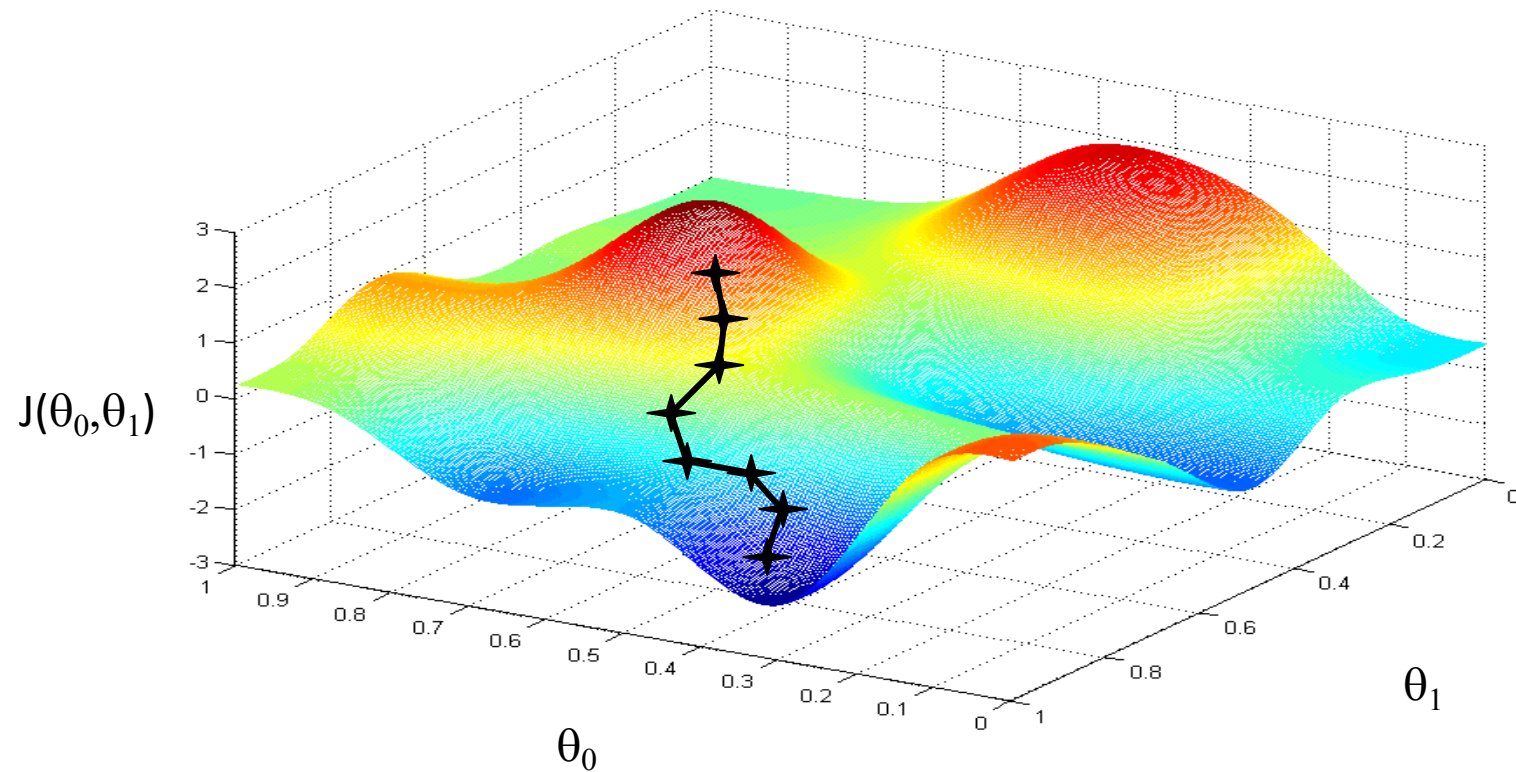
Want $\min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$

Outline:

- Start with some θ_0, θ_1 (common choice $\theta_0 = 0, \theta_1 = 0$)
- Keep changing θ_0, θ_1 to reduce $J(\theta_0, \theta_1)$
- Until we hopefully end up at a minimum

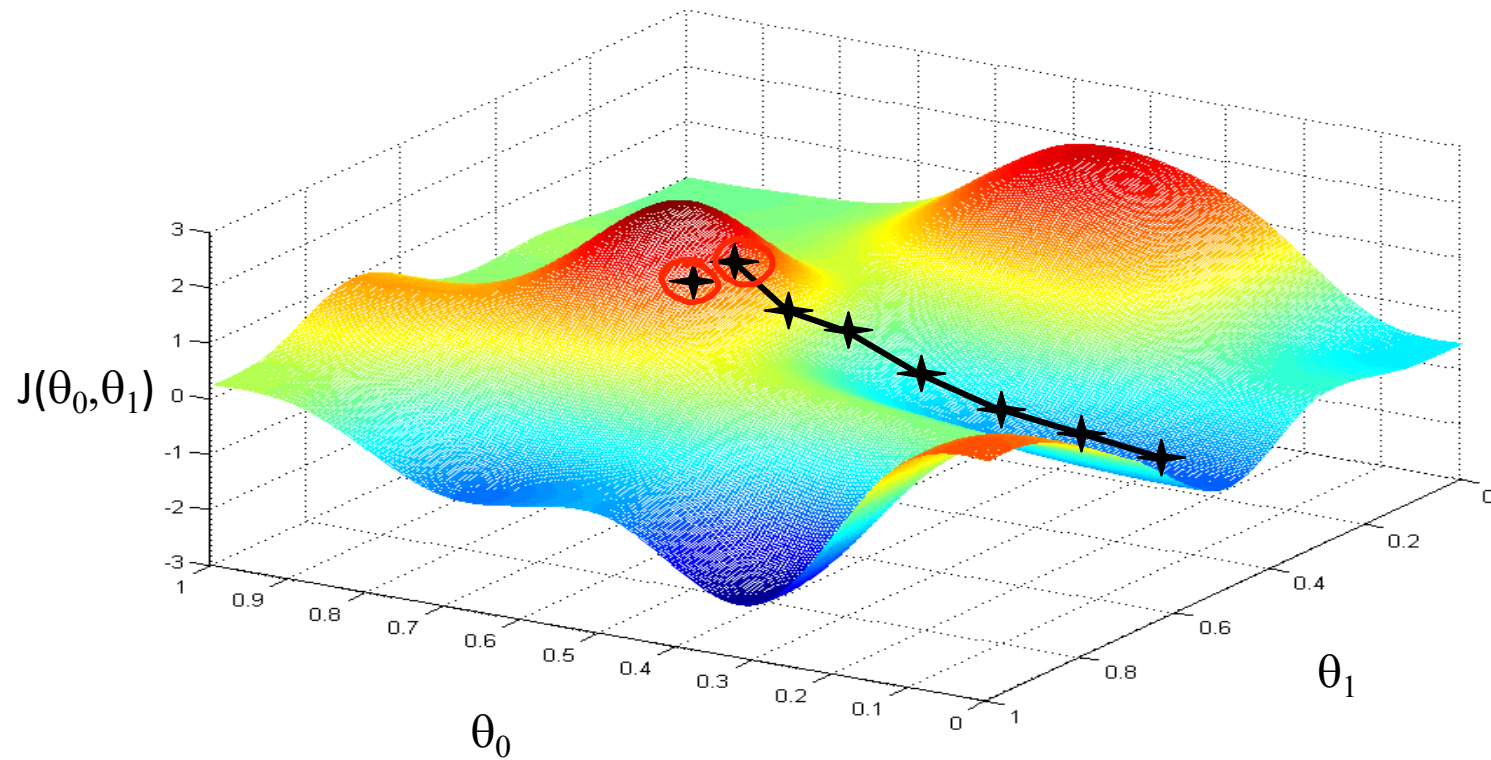
Gradient Descent Algorithm

Here an example of a general and no convex cost function



Gradient Descent Algorithm

Here an example of a general and no convex cost function



Gradient Descent Algorithm

Gradient Descent algorithm

repeat until convergence {
 $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$ (for $j = 0$ and $j = 1$)
}

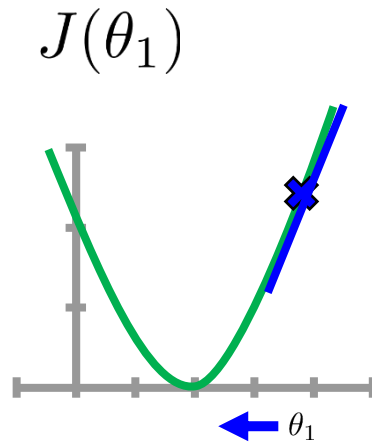
Correct: Simultaneous update

`temp0` := $\theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$
`temp1` := $\theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$
 θ_0 := `temp0`
 θ_1 := `temp1`

Incorrect:

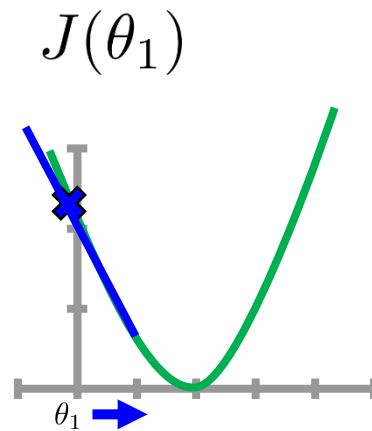
`temp0` := $\theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$
 θ_0 := `temp0`
`temp1` := $\theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$
 θ_1 := `temp1`

Gradient Descent Intuition



$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

$$\frac{\partial}{\partial \theta_1} J(\theta_1) > 0$$



$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

$$\frac{\partial}{\partial \theta_1} J(\theta_1) < 0$$

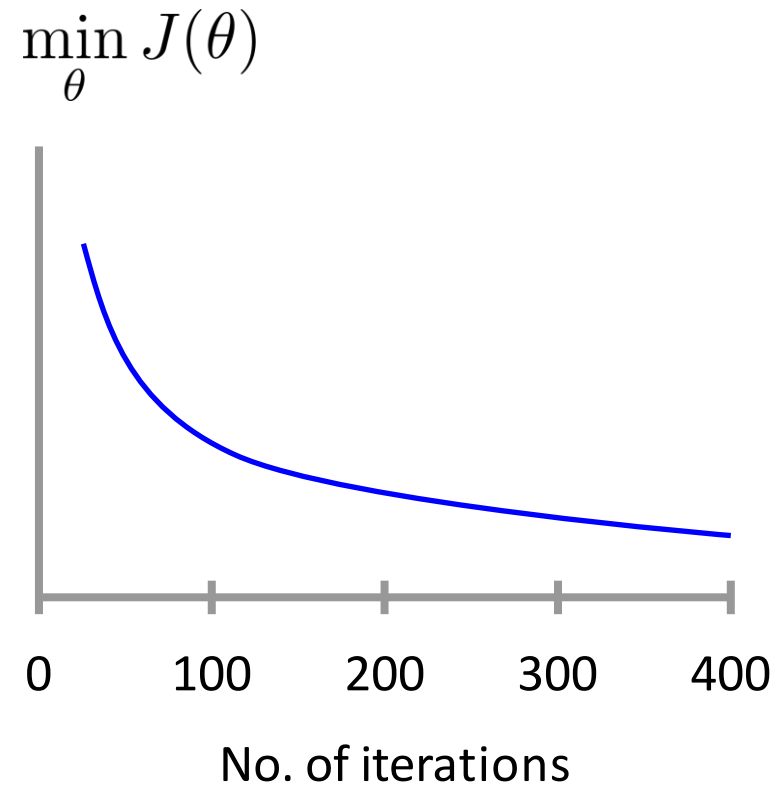
Gradient Descent

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

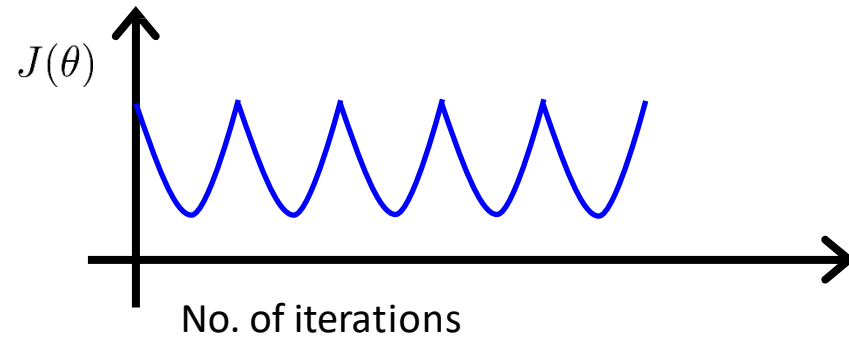
- “Debugging”: How to make sure gradient descent is working correctly.
- How to choose learning rate.

Gradient Descent

Making sure gradient descent is working correctly.



Gradient Descent



Gradient descent not working.

- For sufficiently small α , $J(\theta)$ should decrease on every iterations.
- But if α is too small, gradient descent can be slow to converge.

Gradient Descent

Summary

If α is too small: slow convergence

If α is too large: $J(\theta)$ may not decrease on every iterations; may not converge

To choose α try

..., 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1, ...

Gradient Descent for Linear Regression

Gradient Descent for Linear Regression

Gradient descent algorithm

repeat until convergence {
 $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$
 (for $j = 1$ and $j = 0$)
}

Linear Regression Model

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Gradient Descent for Linear Regression

More details...

$$\begin{aligned}\frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) &= \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 = \\ &= \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^m (\theta_0 + \theta_1 x^{(i)} - y^{(i)})^2\end{aligned}$$

$$j = 0: \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$j = 1: \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

Gradient Descent for Linear Regression

repeat until convergence {

$$\left. \begin{aligned} \theta_0 &:= \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \\ \theta_1 &:= \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)} \end{aligned} \right\}$$

update θ_0 and θ_1 simultaneously

}

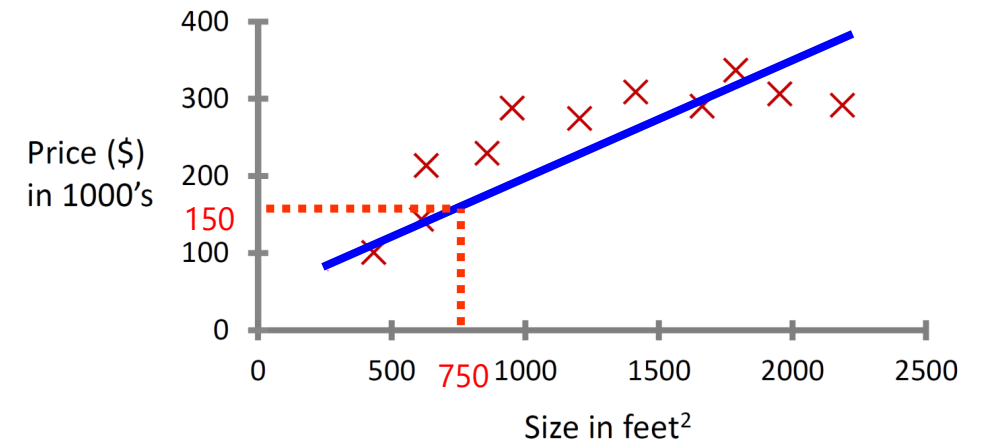
Linear Regression with multiple features (variables)

Linear Regression with one variable

Size in feet ² (x)	Price (\$) in 1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

Hypothesis:

$$h_{\theta}(x_1) = \theta_0 + \theta_1 x_1$$



Linear Regression with multiple features (variables)

Size (feet ²)	Number of bedrooms	Number of floors	Age of home (years)	Price (\$1000)
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...
x_1	x_2	x_3	x_4	y_3

n = Number of features (here $n=4$)

$x^{(i)}$ = input features of i^{th} training example (here $x^{(2)} = [1416 \ 3 \ 2 \ 40]$)

$x_j^{(i)}$ = value of the feature j in i^{th} training example (here $x_3^{(2)} = 2$)

Linear Regression with multiple features (variables)

Hyphothesis with a variable:

$$h_{\theta}(x_1) = \theta_0 + \theta_1 x_1$$

Hyphothesis with multiple variables (e.g. 4)

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4$$

e.g $h_{\theta}(x) = 80 + 0.1x_1 + 0.01x_2 + 3x_3 - 2x_4$

Size (feet ²)	Number of bedrooms	Number of floors	Age of home (years)	Price (\$1000)
2104	5	1	45	460
1416	3	2	40	232

Linear Regression with multiple features (variables)

Hypothesis with multiple variables (e.g. four variables)

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4$$

For convenience of notation $x_0 = 1$. Let's $\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \\ \theta_4 \end{bmatrix}$ $x = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$

The hypothesis can be written in vectorization form: $h_{\theta}(x) = \theta^T x$

Linear Regression with multiple features (variables)

Hypothesis:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$$

Parameters:

$$\theta_0, \theta_1, \dots, \theta_n$$

Cost Function

$$J(\theta_0, \theta_1, \dots, \theta_n) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Goal:

$$\min_{(\theta_0, \theta_1, \dots, \theta_n)} J(\theta_0, \theta_1, \dots, \theta_n)$$

Linear Regression with multiple features (variables)

Gradient Descent

Previously (n=1):

Repeat {

$$\theta_0 := \theta_0 - \alpha \underbrace{\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})}_{\frac{\partial}{\partial \theta_0} J(\theta)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

(simultaneously update θ_0, θ_1)

}

New algorithm ($n \geq 1$):

Repeat {

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(simultaneously update θ_j for
 $j = 0, \dots, n$)

}

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_1^{(i)}$$

$$\theta_2 := \theta_2 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_2^{(i)}$$

...

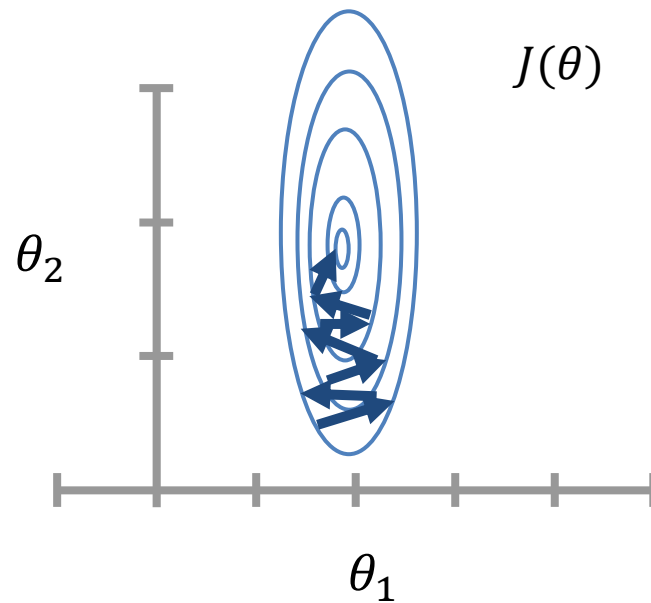
Feature Normalization

Feature Scaling

Idea: Make sure features are on a similar scale

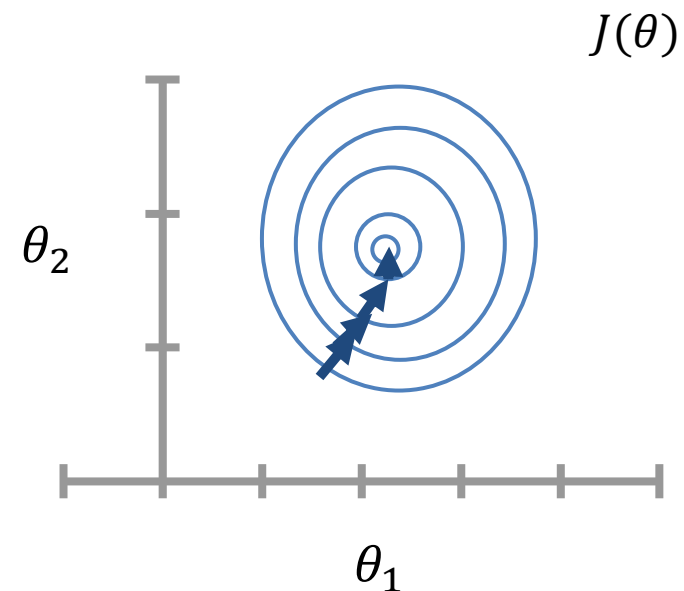
E.g. $x_1 = \text{size (0-2000 feet}^2\text{)}$

$x_2 = \text{number of bedrooms (1-5)}$



$$x_1 = \frac{\text{size (feet}^2\text{)}}{2000}$$

$$x_2 = \frac{\text{number of bedrooms}}{5}$$



Mean Normalization

Replace x_i with $x_i - \mu_i$ to make features have approximately zero mean (Do not apply to $x_0 = 1$)

$$x_1 = \frac{size - 1000}{2000}$$

$$x_2 = \frac{\#bedrooms - 2}{5}$$

$$-0.5 \leq x_1 \leq 0.5, -0.5 \leq x_2 \leq 0.5$$

More formally, $x'_i = \frac{x_i - \mu_i}{\sigma_i}$

where μ_i, σ_i are the mean and standard deviation computed on the training set.

MinMax Normalization

The MinMax normalization shifts the data such that all features are exactly between 0 and 1.

More formally, $x'_i = \frac{x_i - \min(x)}{\max(x) - \min(x)}$

Where x_i is the original value, x'_i is the normalized value.

Mean and MinMax Normalization Comparison

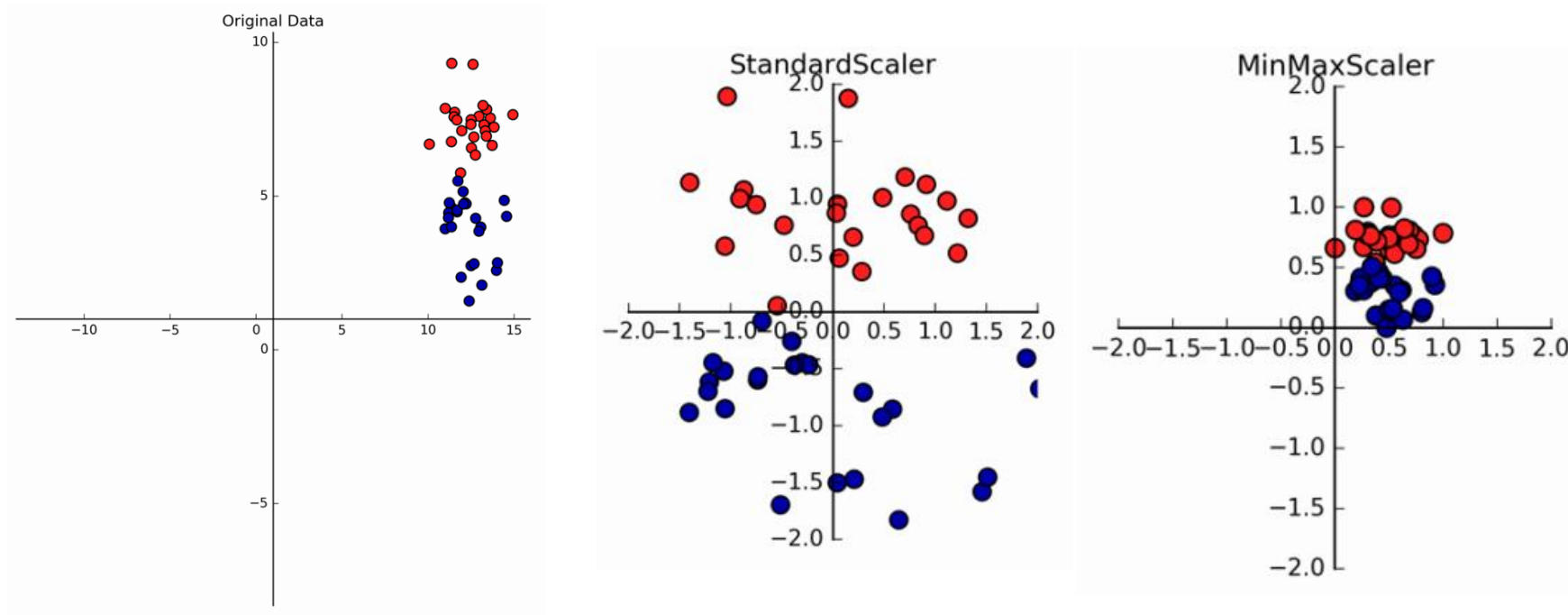
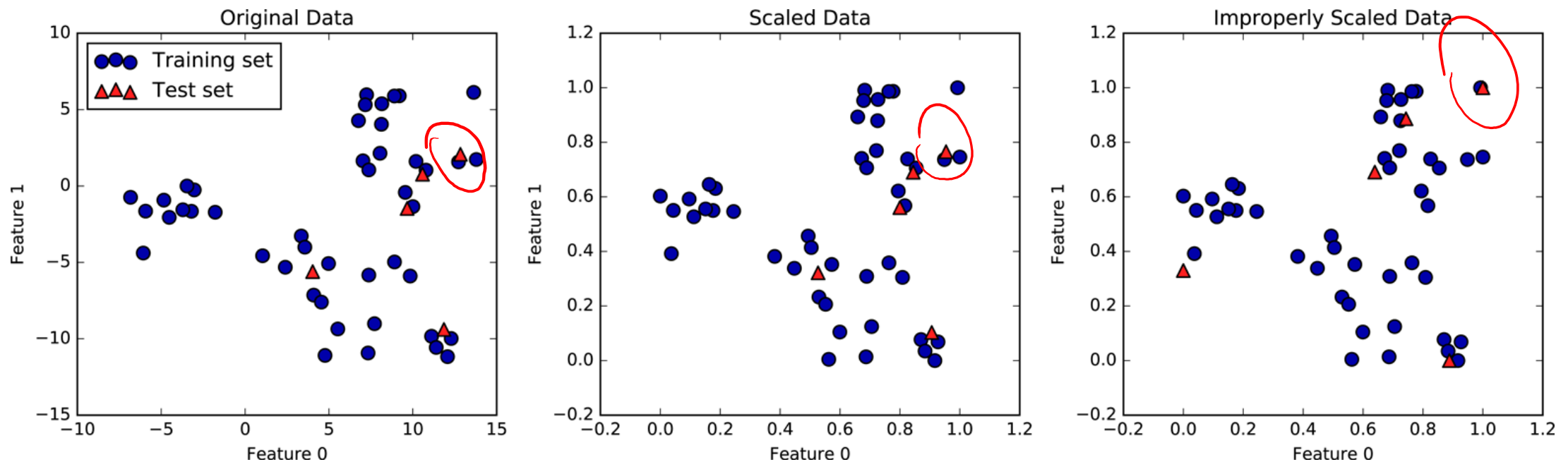


Figure shows a two-class classification dataset with two features and a comparison between Mean Normalization (StandardScaler) and MinMax Normalization (MinMaxScaler)

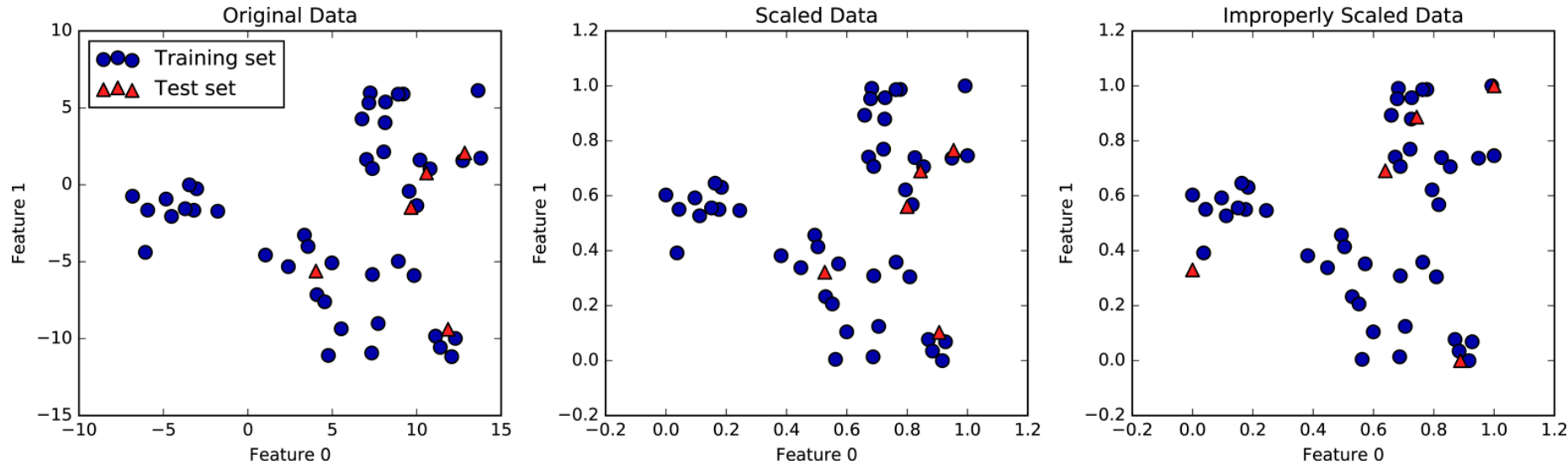
Scaling Training and Test Data the Same Way

It is important to apply exactly the same transformation to the training set and the test set for the supervised model to work on the test set.

The figure illustrates what would happen if we were to use the minimum and range of the test set instead:

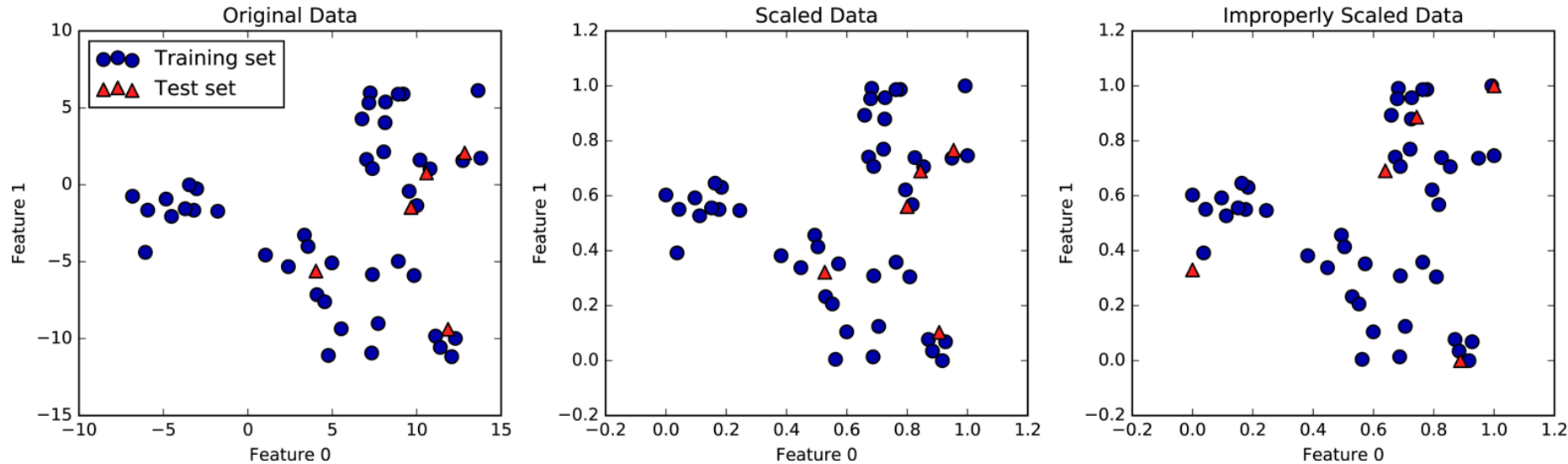


Scaling Training and Test Data the Same Way



Note: The **first panel** is an unscaled two-dimensional dataset, with the training set shown as circles and the test set shown as triangles. The **second panel** is the same data, but scaled using the `MinMaxScaler`. Here, we called `fit` on the training set, and then called `transform` on the training and test sets. You can see that the dataset in the **second panel** looks identical to the first; only the ticks on the axes have changed. Now all the features are between 0 and 1. You can also see that the minimum and maximum feature values for the test data (the triangles) are not 0 and 1.

Scaling Training and Test Data the Same Way



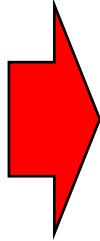
Note: The **third panel** shows what would happen if we scaled the training set and test set separately. In this case, the minimum and maximum feature values for both the training and the test set are 0 and 1. But now the dataset looks different. The test points moved incongruously to the training set, as they were scaled differently. We changed the arrangement of the data in an arbitrary way. Clearly this is not what we want to do.

Scaling Training and Test Data the Same Way: A synthetic Example

We scaled the training set and test set with same transformation (parameter transformation inferred by the training set) - **Correct**

Training Set

X	Y
0	0
5	10
10	20



Training Set

X-Norm	Y
0	0
0.5	10
1	20

Test Set

X	Y
10	20
15	30
20	40

MinMax feature
Normalization

Test Set

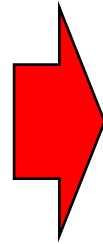
X-Norm	Y
1	20
1.5	30
2	40

Scaling Training and Test Data the Same Way: A synthetic Example

We scaled the training set and test set separately - **Incorrect**

Training Set

X	Y
0	0
5	10
10	20



Training Set

X-Norm	Y
0	0
0.5	10
1	20

Test Set

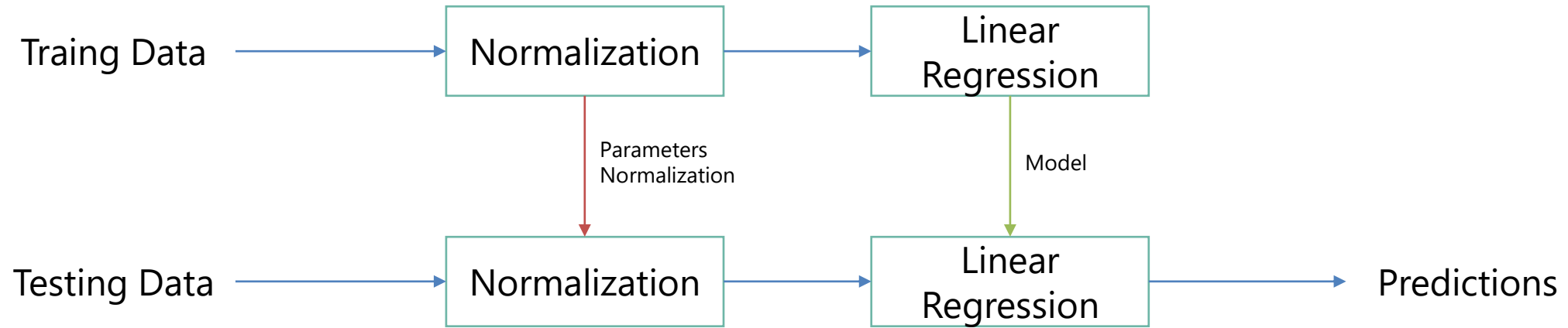
X	Y
10	20
15	30
20	40

MinMax feature
Normalization

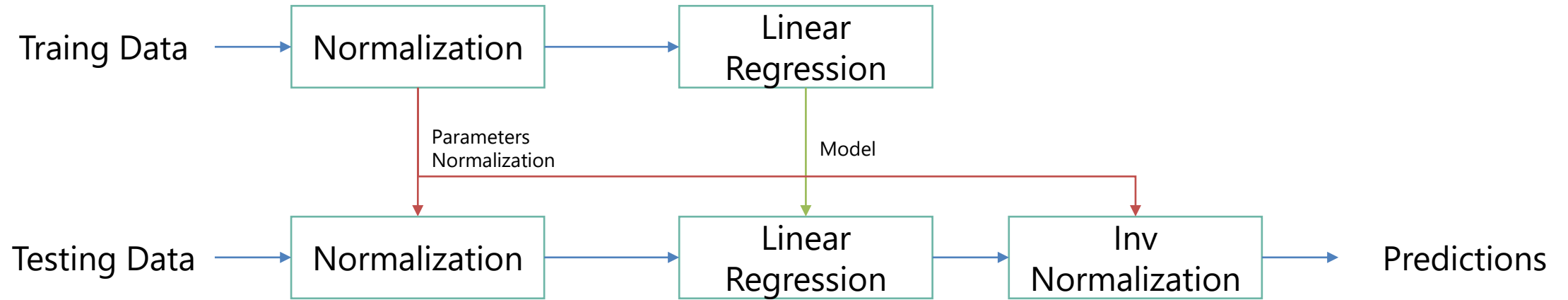
Test Set

X-Norm	Y
0	20
0.5	30
1	40

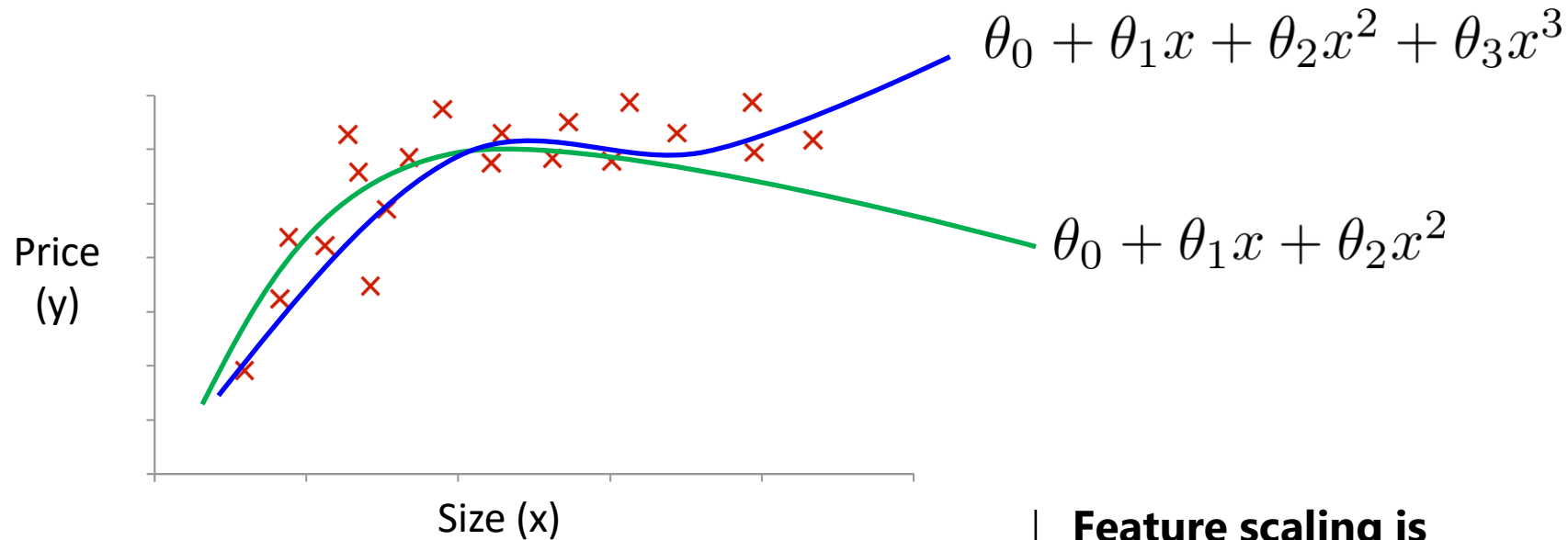
Pipeline #1



Pipeline #2



Polynomial regression



$$\begin{aligned}h_{\theta}(x) &= \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 \\&= \theta_0 + \theta_1(\text{size}) + \theta_2(\text{size})^2 + \theta_3(\text{size})^3\end{aligned}$$

$$\begin{aligned}x_1 &= (\text{size}) \\x_2 &= (\text{size})^2 \\x_3 &= (\text{size})^3\end{aligned}$$

Feature scaling is important

x_1 range 1 – 1000
 x_2 range 1 – 1000000
 x_3 range 1 – 10^9

Pearson correlation coefficient

Pearson correlation coefficient

$$\rho = \frac{1}{N-1} \sum_{i=1}^N \left(\frac{X_i - \mu_X}{\sigma_X} \right) \left(\frac{Y_i - \mu_Y}{\sigma_Y} \right)$$

where μ_X and σ_X are the mean and the standard deviation of X respectively and μ_Y and σ_Y are the mean and standard deviation of Y

$$\mu_X = \frac{1}{N} \sum_{i=1}^N X_i$$

Standard Deviation

$$\sigma_X = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (X_i - \mu_X)^2}$$

Pearson correlation coefficient

